Hybrid Multiagent System Architecture for Diabetes Decision Support

Integrating Deep Learning, Knowledge Graph/RAG,
Rule-Based Reasoning, and Reinforcement Learning

PTIT – Intelligent System Development Project

November 2026

Abstract

This document presents the architecture of a hybrid Multiagent System (MAS) for diabetes decision support.

The system builds upon two existing student-developed components: (i) a Deep Learning (DL) model for diabetes prediction and (ii) a diabetes Knowledge Graph and Retrieval-Augmented Generation (RAG) system implemented using Neo4j and the `DB_diabet.cypher` database initialization script.

The proposed system consists of specialized agents for patient data validation, DL prediction, rule-based reasoning, knowledge retrieval, explanation, monitoring, and coordination. An optional Reinforcement Learning (RL) agent is introduced as an advanced extension operating in a simulated environment.

The document defines the system layers, agent responsibilities, communication interfaces, mathematical abstractions, interaction workflow, and development roadmap. It is intended as a technical specification for student implementation and research.

The system is an educational and research prototype. It is not a clinically validated diagnostic or treatment system.

Contents

1 Introduction **3**
- 1.1 Project Background . . . 3
- 1.2 Project Objectives . . . 3

2 Overall System Architecture **3**
- 2.1 Layered Architecture . . . 3
- 2.2 Architecture Diagram . . . 4
- 2.3 Architecture Interpretation . . . 4

3 Agent Types and Responsibilities **5**
- 3.1 Agent Classification . . . 5
- 3.2 Agent Design Principles . . . 5

4 Mathematical Abstraction of the MAS **5**
- 4.1 Multiagent System Definition . . . 5
- 4.2 Agent Input and Output . . . 6

5 Agent Interaction Workflow **6**
- 5.1 Sequential Workflow . . . 6
- 5.2 Sequence Diagram . . . 7
- 5.3 Coordinator Workflow . . . 7

6 Communication and Data Contracts **7**
6.1 Agent Message Structure 7
6.2 Example Message 8
6.3 Agent Interface Design 8

7 Knowledge and Model Integration **8**
7.1 Deep Learning Integration 8
7.2 Knowledge Graph and RAG Integration 9

8 Reinforcement Learning Extension **9**
8.1 Role of the RL Agent 9

9 Implementation Roadmap **10**

10 Safety and Research Limitations **10**

11 Conclusion **10**

1 Introduction

1.1 Project Background

Students have already developed the following components:

1. A Deep Learning model for diabetes prediction.
2. A Knowledge Graph and RAG system for diabetes knowledge retrieval using Neo4j.
3. A graph database initialized from `DB_diabet.cypher`.

The next development stage is to integrate these components into a hybrid Multiagent System.

The proposed system combines several AI paradigms:

- Deep Learning for prediction.
- Knowledge Graph and RAG for knowledge retrieval.
- Rule-Based Reasoning for transparent validation and predefined decision logic.
- Large Language Models for natural-language explanation.
- Reinforcement Learning for adaptive strategies in a simulated environment.
- Multiagent coordination for task orchestration.

1.2 Project Objectives

The main objectives are:

1. Design a modular multiagent architecture.
2. Reuse the existing DL model without rebuilding it.
3. Reuse the existing Neo4j Knowledge Graph.
4. Define explicit agent responsibilities and interfaces.
5. Implement agent communication and coordination.
6. Develop a structured decision-support report.
7. Evaluate individual agents and the complete MAS.
8. Provide a foundation for future research in intelligent multiagent systems.

2 Overall System Architecture

2.1 Layered Architecture

The system is organized into six layers.

Layer 1. User Interaction Layer

Provides interfaces for patients, doctors, and students.

Layer 2. API and Data Processing Layer

Receives requests, validates input data, and prepares patient features.

Layer 3. Multiagent Coordination Layer

Coordinates agent execution, task assignment, and output aggregation.

Layer 4. Intelligent Agent Layer

Contains specialized DL, rule-based, KG/RAG, explanation, monitoring, and RL agents.

Layer 5. Knowledge and Model Layer

Contains the trained DL model, Neo4j Knowledge Graph, RAG retrieval components, and simulation environment.

Layer 6. Evaluation and Audit Layer

Records execution traces, model versions, agent outputs, and experimental metrics.

2.2 Architecture Diagram

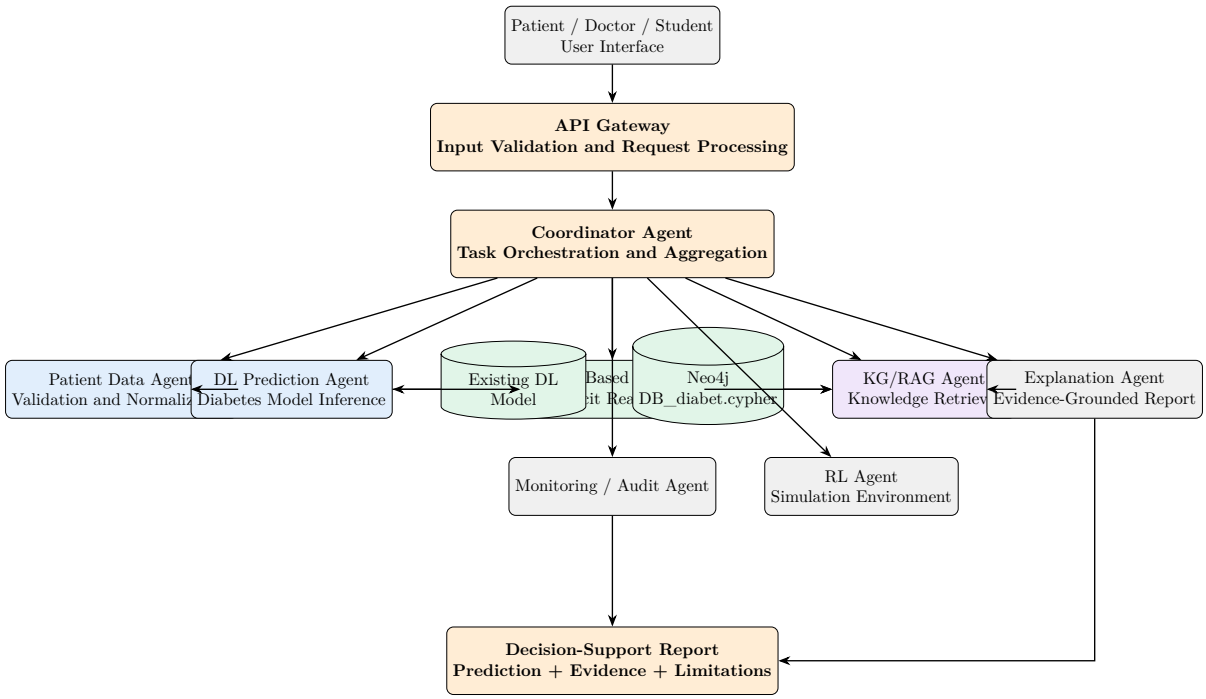


Figure 1: Hybrid Multiagent System architecture for diabetes decision support.

2.3 Architecture Interpretation

The Coordinator Agent is the central orchestration component.

It does not replace the intelligence of specialized agents. Instead, it manages their execution and integrates their results.

The DL Prediction Agent is responsible for inference using the existing trained model.

The KG/RAG Agent retrieves diabetes-related information from the Neo4j knowledge base.

The Rule-Based Agent executes explicit, reviewed rules.

The Explanation Agent constructs a structured report from the outputs of the other agents.

The RL Agent is an optional extension and initially operates only in a simulation environment.

3 Agent Types and Responsibilities

3.1 Agent Classification

Agent	AI Paradigm	Main Responsibility
Patient Data Agent	Deterministic processing	Validates features, data types, missing values, and input consistency.
DL Prediction Agent	Deep Learning	Runs the existing trained diabetes prediction model.
Rule-Based Agent	Symbolic reasoning	Applies explicit rules and produces auditable flags.
KG/RAG Agent	Knowledge Graph and RAG	Retrieves diabetes knowledge and source evidence.
Coordinator Agent	Multiagent orchestration	Schedules agents, aggregates outputs, and manages failures.
Explanation Agent	LLM or template-based reasoning	Produces a grounded explanation of the combined results.
Monitoring Agent	Monitoring and audit	Tracks execution, errors, versions, and performance.
RL Agent	Reinforcement Learning	Learns a policy in a synthetic simulation environment.

Table 1: Agent types and responsibilities.

3.2 Agent Design Principles

Each agent should satisfy the following principles:

1. **Single responsibility:** each agent performs a clearly defined task.
2. **Modularity:** an agent can be implemented and tested independently.
3. **Explicit interfaces:** inputs and outputs are structured and documented.
4. **Traceability:** important outputs include evidence or rationale.
5. **Failure awareness:** errors are returned explicitly rather than hidden.
6. **Extensibility:** agents can be replaced or extended without redesigning the complete system.

4 Mathematical Abstraction of the MAS

4.1 Multiagent System Definition

Let the complete multiagent system be represented by

$$\mathcal{M} = (\mathcal{A}, \mathcal{S}, \mathcal{O}, \mathcal{C}, \mathcal{F}),$$

where:

- $\mathcal{A} = \{A_1, A_2, \ldots, A_n\}$ is the set of agents.
- $\mathcal{S}$ is the global system state space.
- $\mathcal{O}$ is the set of agent observations.

- $\mathcal{C}$ is the communication mechanism.
- $\mathcal{F}$ is the coordination and aggregation function.

For the proposed system, define

$$\mathcal{A} = \{A_{\text{data}}, A_{\text{DL}}, A_{\text{rule}}, A_{\text{KG}}, A_{\text{coord}}, A_{\text{exp}}, A_{\text{monitor}}, A_{\text{RL}}\}.$$

The RL agent may be omitted in the initial implementation.

4.2 Agent Input and Output

Each agent A_i can be modeled as a function

$$A_i : \mathcal{X}_i \to \mathcal{Y}_i,$$

where $\mathcal{X}_i$ is its input space and $\mathcal{Y}_i$ is its output space.
For example,

$$A_{\text{DL}} : \mathcal{X}_{\text{patient}} \to \mathcal{Y}_{\text{prediction}}.$$

The Knowledge Agent is represented by

$$A_{\text{KG}} : \mathcal{Q} \to \mathcal{E},$$

where $\mathcal{Q}$ is the query space and $\mathcal{E}$ is the retrieved evidence space.
The coordinator integrates outputs through

$$\mathcal{F}\left(y_{\text{DL}}, y_{\text{rule}}, y_{\text{KG}}, y_{\text{monitor}}\right) = y_{\text{system}}.$$

This aggregation function should preserve the distinction between model prediction, rule results, and retrieved evidence.

5 Agent Interaction Workflow

5.1 Sequential Workflow

The initial implementation follows this workflow:

1. A user submits patient features and an optional question.
2. The API receives the request.
3. The Patient Data Agent validates the input.
4. If validation fails, the Coordinator terminates the workflow and returns an error.
5. The DL Prediction Agent executes the existing model.
6. The Rule-Based Agent evaluates predefined rules.
7. The KG/RAG Agent retrieves relevant knowledge.
8. The Coordinator aggregates agent outputs.
9. The Explanation Agent generates a structured report.
10. The Monitoring Agent records execution metadata.
11. The report is returned to the user.

5.2 Sequence Diagram

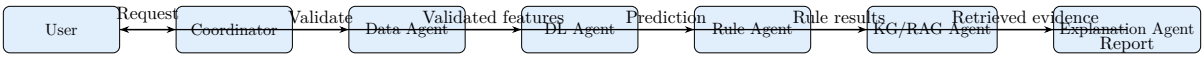


Figure 2: Simplified sequential agent interaction workflow.

5.3 Coordinator Workflow

The coordinator should implement the following logical algorithm.

1. Receive the request.
2. Generate a request identifier.
3. Invoke the Data Agent.
4. Check the validation status.
5. If invalid, return a validation error.
6. Invoke the DL Agent.
7. Invoke the Rule Agent.
8. Invoke the KG/RAG Agent.
9. Collect outputs.
10. Invoke the Explanation Agent.
11. Record the execution trace.
12. Return the structured result.

6 Communication and Data Contracts

6.1 Agent Message Structure

Agents should exchange structured messages.

A general message is represented as

$$m_i = (id, agent, status, payload, evidence, version)\,.$$

The fields are:

- *id*: request identifier.
- *agent*: agent name.
- *status*: execution status.
- *payload*: structured output.
- *evidence*: retrieved sources or rationale.
- *version*: model or agent version.

6.2 Example Message

```
{
  "request_id": "req_001",
  "agent": "DiabetesPredictionAgent",
  "status": "ok",
  "payload": {
    "prediction": 1,
    "score": 0.78
  },
  "evidence": [],
  "model_version": "diabetes_dl_v1"
}
```

The numerical score above is only an illustrative example. Its interpretation depends on the trained model and calibration.

6.3 Agent Interface Design

Each agent should expose a method with a documented contract.

Agent	Input	Output
Data Agent	Raw patient features	Validated features or errors
DL Agent	Validated features	Prediction and model metadata
Rule Agent	Features and prediction	Rule results and review flags
KG/RAG Agent	Natural-language query	Retrieved evidence and sources
Coordinator	User request	Aggregated workflow result
Explanation Agent	Prediction, rules, evidence	Structured explanation
Monitoring Agent	Agent execution metadata	Audit record
RL Agent	Simulation state	Simulation action

Table 2: Agent interfaces.

7 Knowledge and Model Integration

7.1 Deep Learning Integration

The existing DL model is treated as a reusable service.

$$\text{Patient Features} \longrightarrow \text{DL Model} \longrightarrow \text{Prediction Output}.$$

The students should document:

- model architecture;
- training dataset;
- feature names and order;
- preprocessing method;
- output format;
- model version;
- evaluation metrics.

The MAS should not assume that the model is clinically validated simply because its test accuracy is high.

7.2 Knowledge Graph and RAG Integration

The existing knowledge repository is

$$\mathcal{K} = (G_{\text{Neo4j}}, D_{\text{RAG}}),$$

where:

- G_{Neo4j} is the diabetes Knowledge Graph;
- D_{RAG} is the document or text retrieval collection, if implemented.

The Knowledge Agent performs:

1. Query interpretation.
2. Entity identification.
3. Graph retrieval.
4. Relevant document retrieval.
5. Evidence ranking.
6. Source attribution.

Retrieved knowledge should be presented as supporting evidence rather than as an independent clinical diagnosis.

8 Reinforcement Learning Extension

8.1 Role of the RL Agent

The RL Agent is an optional advanced module.

The initial MAS does not require RL to perform prediction or clinical decision-making.

The RL agent can be developed in a synthetic simulation environment for educational or monitoring-policy experiments.

The Markov Decision Process is defined by

$$\mathcal{M}_{\text{RL}} = (S, A, P, R, \gamma),$$

where:

- S: simulated state space;
- A: permitted simulation action space;
- $P(s'|s, a)$: transition probability;
- $R(s, a, s')$: reward function;
- γ: discount factor.

The RL agent seeks a policy

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{T} \gamma^t r_t \right].$$

Possible simulation actions include selecting an educational module or choosing a simulated follow-up interval.

The RL policy must not autonomously prescribe medication, change treatment, or control care for real patients.

9 Implementation Roadmap

Phase	Development Task	Expected Deliverable
1	Integrate existing DL model	Diabetes Prediction Agent
2	Integrate Neo4j KG/RAG	Knowledge Retrieval Agent
3	Implement input validation	Patient Data Agent
4	Implement explicit rules	Rule-Based Agent
5	Implement orchestration	Coordinator Agent
6	Implement explanation	Explanation Agent
7	Develop user interface	Working MAS prototype
8	Perform testing and evaluation	Experimental report
9	Develop RL simulation	Optional advanced agent

10 Safety and Research Limitations

The project is an educational and research prototype.

The following requirements apply:

- The model output must not be presented as a confirmed diagnosis.
- The system must not prescribe medication or dosage.
- The RL agent must operate in simulation only.
- Clinical rules require review by qualified professionals before any clinical use.
- The system should use synthetic, public, or properly approved de-identified data.
- Retrieved KG/RAG content must be traceable to sources.
- Uncertain or conflicting results should be flagged for qualified human review.

11 Conclusion

This architecture provides a modular foundation for developing a Hybrid Multiagent System for Diabetes Decision Support.

The system integrates existing Deep Learning and Knowledge Graph/RAG components with rule-based reasoning, agent coordination, and evidence-grounded explanation.

The proposed architecture supports incremental development, independent agent testing, reproducible evaluation, and future research extensions.

The first implementation should focus on the Data Agent, DL Prediction Agent, KG/RAG Agent, Rule-Based Agent, and Coordinator Agent.

The RL Agent is introduced only after the integrated system has been implemented and evaluated.